



Ludwig moves to Youtube

Twitch streamer Ludwig Ahgren is switching platforms and moving to YouTube.
<https://dgit.in/dec21-05>



AMC to handout NFTs

AMC and Sony will hand out NFTs to 'Spider-Man' advance ticket buyers.
<https://dgit.in/dec21-06>

Creating a Task Component

Now, let us go ahead and create a Task.jsk file in the UI folder which will represent a single task from the To-Do list. Create the file and add the following code:

```
import React from 'react';

export const Task = ({ task }) => {
  return <li>{task.text}</li>
};
```

Let us now open up our App.jsx file in imports/ui and create some placeholder tasks for the time being. These will eventually be fetched from our database.

```
import React from 'react';

const tasks = [
  { _id: 1, text: 'Learn Meteor' },
  { _id: 2, text: 'Create a Simple Application' },
  { _id: 3, text: 'Deploy it on Meteor Cloud' },
];
```

Now we can iterate over these tasks with the map function as tasks is an Array object. We can render it out onto the app as follows:

```
export const App = () => (
  <div>
    <h1>Welcome to my first Meteor App!</h1>

    <ul>
      { tasks.map(task =>
        <Task key={ task._id } task={ task } /> ) }
    </ul>
  </div>
);
```

So all in all our imports/ui/App.jsx file looks as follows:

```
import React from 'react';
```

```
const tasks = [
  { _id: 1, text: 'Learn Meteor' },
  { _id: 2, text: 'Create a Simple Application' },
  { _id: 3, text: 'Deploy it on Meteor Cloud' },
];

export const App = () => (
  <div>
    <h1>Welcome to my first Meteor App!</h1>

    <ul>
      { tasks.map(task =>
        <Task key={ task._id } task={ task } /> ) }
    </ul>
  </div>
);
```

Add the key property to the task to avoid warnings from React. We can also safely remove the Hello and Info from our App component, their imports, and the Hello.jsx and Info.jsx files as well.

Setting up Database Collections

When the project files were created by Meteor, MongoDB was also automatically set up. We can now set up a collection, which will hold our tasks.

We can create a new collection to store our tasks by creating a new file at imports/api/TasksCollection.js, and adding the following code:

```
import { Mongo } from 'meteor/mongo';

export const TasksCollection = new Mongo.Collection('tasks');
```

We used the folder api simply because it's a good location for files related to making api calls.

Now, we can update server/main.js by importing in our TasksCollection, with the following code:

```
import { Meteor } from 'meteor/meteor';
```

```
import { TasksCollection }
from '/imports/api/TasksCollection';
```

```
const insertTask = taskText => TasksCollection.insert({
  text: taskText });
```

```
Meteor.startup(() => {
  if (TasksCollection.find().count() === 0) {
    [
      //add list entries here
    ].forEach(insertTask)
  }
});
```

Now, we can go back to App.jsx in the ui folder and update it with the useTracker function which allows reactivity via a React Hook in our components.

This will refresh every time the data changes on the backend, something Meteor was geared to handle from the very beginning. So the updated code looks as follows:

```
import React from 'react';
import { useTracker } from 'meteor/react-meteor-data';
import { TasksCollection } from '/imports/api/TasksCollection';
import { Task } from './Task';
```

```
export const App = () => {
  const tasks = useTracker(() => TasksCollection.find({}).fetch());
```

```
  return (
    <div>
      <h1>Welcome to my first Meteor App!</h1>
```

```
      <ul>
        { tasks.map(task => <Task key={ task._id } task={ task } /> ) }
      </ul>
    </div>
  );
```